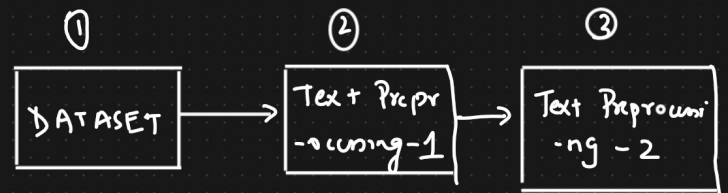


Text Preprocessing → What we have learnt?

Sentiment Analysis

	<u>Text</u>	<u>O/p</u>
D1	The food is good	1
D2	The food is bad	0
D3	Pizza is Amazing	1
D4	Burger is bad	0



- ① Tokenization
- ② Lowercase the words
- ③ Regular Expression
- ① STEMMING
- ② LEMMATIZATION
- ③ STOPWORDS



D1 → [1 0 1 1]

Gensim

- ① One Hot Encoding
- ② Bag of Words (BOW)
- ③ TF-IDF
- ④ Word2Vec
- ⑤ Avg Word2Vec

① One Hot Encoding

Vocabulary size = 7

Vocabulary {unique words}

The	food	is	good	bad	Pizza	Amazing
1	0	0	0	0	0	0
0	1	0	0	0	0	0

	<u>Text</u>	<u>O/p</u>
D1	The food is good	1
D2	The food is bad	0
D3	Pizza is Amazing	1

Test [Burger is bad]

D1 [[1 0 0 0 0 0 0],
[0 1 0 0 0 0 0],
4x7 [0 0 1 0 0 0 0].

D2 [[1 0 0 0 0 0 0],
[0 1 0 0 0 0 0],
4x7 [0 0 1 0 0 0 0],

D3 [[0 0 0 0 0 1 0],
[0 0 1 0 0 0 0],
3x7 [0 0 0 0 0 0 1]

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

{SOK vocabulary size}

Advantages

① Easy to implement with python

[sklearn OneHotEncoder, pd.get-dummies()]

Disadvantages

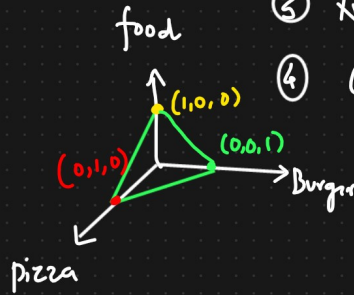
① Sparse matrix → Overfitting

② ML Algorithm → Fixed Size I/P

③ No semantic meaning is getting captured

④ Out of Vocabulary (OOV).

food	pizza	burger
[1	0	0]
[0	1	0]
[0	0	1]



② Bag of Words

Dataset

Text	O/P
He is a good boy	1
She is a good girl	1
Boy and girl are good	1

Lower all the words case
⇒
Stopwords

S1 → good boy
S2 → good girl good
S3 → Boy girl good ~~School~~

Test

<u>Vocabulary</u>	<u>frequency</u>		[good	boy	girl]	<u>O/p</u>	
good	3	⇒	S1	[1	1	0]	1
boy	2		S2	[1	0	1]	1
girl	2		S3	[1	1	1]	1

Binary Bow and Bow

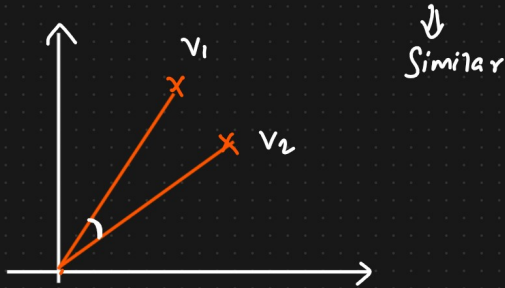
{ 1 and 0 }

{ Count will get updated based on frequency }

Advantages

- ① Simple and Intuitive
- ② Fixed Sized I/p $\rightarrow$ ML Algorithms

$\left\{ \begin{array}{l} \text{The food is good} \rightarrow [1 \ 1 \ 1 \ 0 \ 1] \rightarrow v_1 \\ \text{The food is not good} \rightarrow [1 \ 1 \ 1 \ 1 \ 1] \rightarrow v_2 \end{array} \right.$



Disadvantages

- ① Sparse matrix or array $\rightarrow$ Overfitting
- ② Ordering of the word is getting changed
- ③ **Out of Vocabulary (OOV)**.
- ④ Semantic meaning is still not captured.

② N-grams Eg: bigrams, trigrams

$S_1 \rightarrow \text{The food is good}$
 $S_2 \rightarrow \text{The food is not good}$

Bigram

	food	not	good
$\rightarrow$	1	0	1
$\rightarrow$	1	1	1

	food	not	good	food good	food not	not good
S_1	1	0	1	1	0	0
S_2	1	1	1	0	1	1

Sklearn $\rightarrow$ n-grams = (1, 1) $\rightarrow$ unigrams

= (1, 2) $\rightarrow$ unigram, bigram

= (1, 3) $\rightarrow$ unigram, bigram, trigram

= (2, 3) $\rightarrow$ Bigram, trigram.

④ TF-IDF [Term Frequency - Inverse Document Frequency]

S1 → good boy

S2 → good girl

S3 → boy girl good

$$\text{Term Freq (TF)} = \frac{\text{No. of rep of words in sentence}}{\text{No. of words in sentence}}$$

$$\text{IDF} = \log_e \left(\frac{\text{No. of sentences}}{\text{No. of sentences containing the word}} \right)$$

	<u>Term Frequency</u>			*	<u>IDF</u>	
	S1	S2	S3	Words	IDF	
good	1/2	1/2	1/3	good	$\log_e(3/3) = 0$	
boy	1/2	0	1/3	boy	$\log_e(3/2)$	
girl	0	1/2	1/3	girl	$\log_e(3/2)$	

Final TF-IDF

	[good boy girl]			<u>Ops</u>	<u>Bow</u>		
Sent 1	0	$\frac{1}{2} \times \log_e(3/2)$	0		1	1	0
Sent 2	0	0	$\frac{1}{2} \log_e(3/2)$		1	0	1
Sent 3	0	$\frac{1}{3} \log_e(3/2)$	$\frac{1}{3} \log_e(3/2)$		1	1	1

Advantages

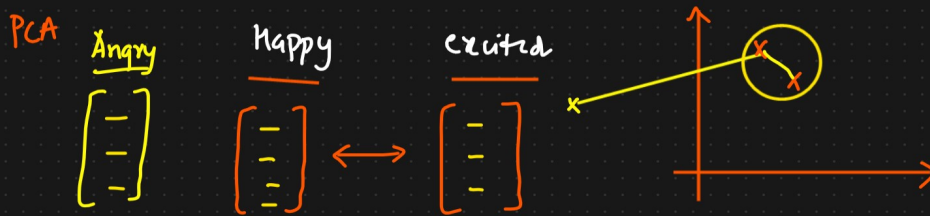
- ① Intuitive
- ② Fixed Size → Vocab size
- ③ Word Importance is getting Captured

Disadvantages

- ① Sparsity still exists
- ② OOV

Word Embeddings [Wikipedia]

In natural language processing (NLP), word embedding is a term used for the representation of words for text analysis, typically in the form of a real-valued vector that encodes the meaning of the word such that the words that are closer in the vector space are expected to be similar in meaning.



Words → vectors

Sentence → vectors

Word Embeddings

ANN

Count or frequency

Deep Learning Trained Model

- ① ONE
- ② BOW
- ③ TF-IDF

Word2Vec

CBOW

Skipgram

[Continuous BAG OF WORDS]

Word2Vec → Feature Representation

Google

Word2vec is a technique for natural language processing published in 2013. The word2vec algorithm uses a neural network model to learn word associations from a large corpus of text. Once trained, such a model can detect synonymous words or suggest additional words for a partial sentence. As the name implies, word2vec represents each distinct word with a particular list of numbers called a vector.

Vocabulary → Unique words → Corpus.

	Boy	Girl	KING	QUEEN	Apple	Mango
Gender	-1	1	-0.92	+0.93	0.01	0.05
Royal	0.01	0.02	0.95	0.96	-0.02	0.02
Age	0.03	0.02	0.75	0.68	0.95	0.96
Food	-	-	-	-	0.91	0.92

300 dimension
with

$\begin{bmatrix} - \\ - \end{bmatrix}$

$$[KING - BOY + QUEEN = GIRL]$$

$$KING [0.95, 0.96]$$

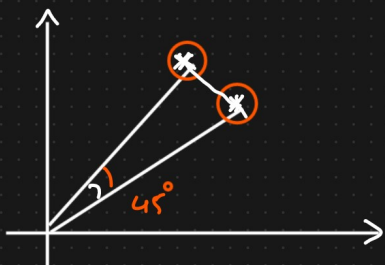
$$MAN [0.95, 0.98]$$

$$QUEEN [-0.96, 0.95]$$

$$WOMEN [-0.94, -0.96]$$

$$KING - MAN + QUEEN = WOMEN$$

Cosine Similarity



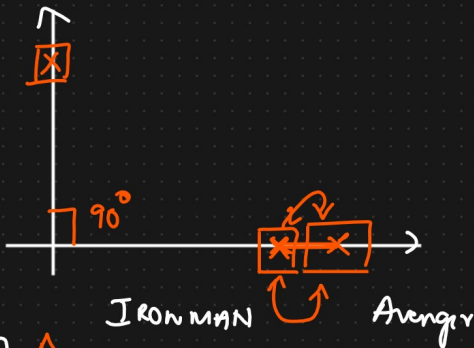
$$Distance = 1 - \text{Cosine Similarity}$$

$$\text{Cosine-Sim} = \cos 45^\circ = \frac{1}{\sqrt{2}} = 0.7071$$

$$Distance = 1 - 0.7071$$

$$\rightarrow = 0.29$$

$$\boxed{1-1=0}$$



$$Distance = 1 - 0$$

$$= 1$$

$$Distance = 1 - \cos 0$$

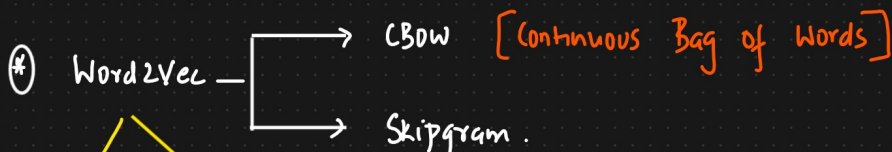
$$= 1 - 1$$

$$= 0\%$$

Action
Comic
Comedy



ANN, Loss, Optimizers



Pretrained
Model

Train A
Model For Scratch

① CBOW [Continuous Bag of Words]

CORPUS ÷ DATASET

[inNeuron Company IS Related To DATA SCIENCE] → Vocabulary

300 dimension 300 ← Window size = 5 ⇒ Feature Representation ONE

I/p

O/p

→ [inNeuron, Company, Related, To]

IS

inNeuron [1 0 0 0 0 0 0]

→ Company, IS, To, DATA

Related

Company [0 1 0 0 0 0 0]

→ IS, Related, DATA, SCIENCE

To

Related [0 0 0 1 0 0 0]

to [0 0 0 0 1 0 0]

I/p Layer

ANN

CBOW

{ Fully Connected Neural N/w }

Word → vector

O/p = []

inNeuron

1
0
0
0
0
0
0

7x5

Initialize weights

HL1

Window

O/p Layer

Company

0
1
0
0
0
0
0

7x5

Window

Window

Related

0
0
0
1
0
0
0

7x5

Window

Window

To

0
0
0
0
1
0
0

7x5

Window

Window

Backward Propagation

$\hat{y} - y \rightarrow \text{Loss W}$

0.25
0.33
↓ + Minimal

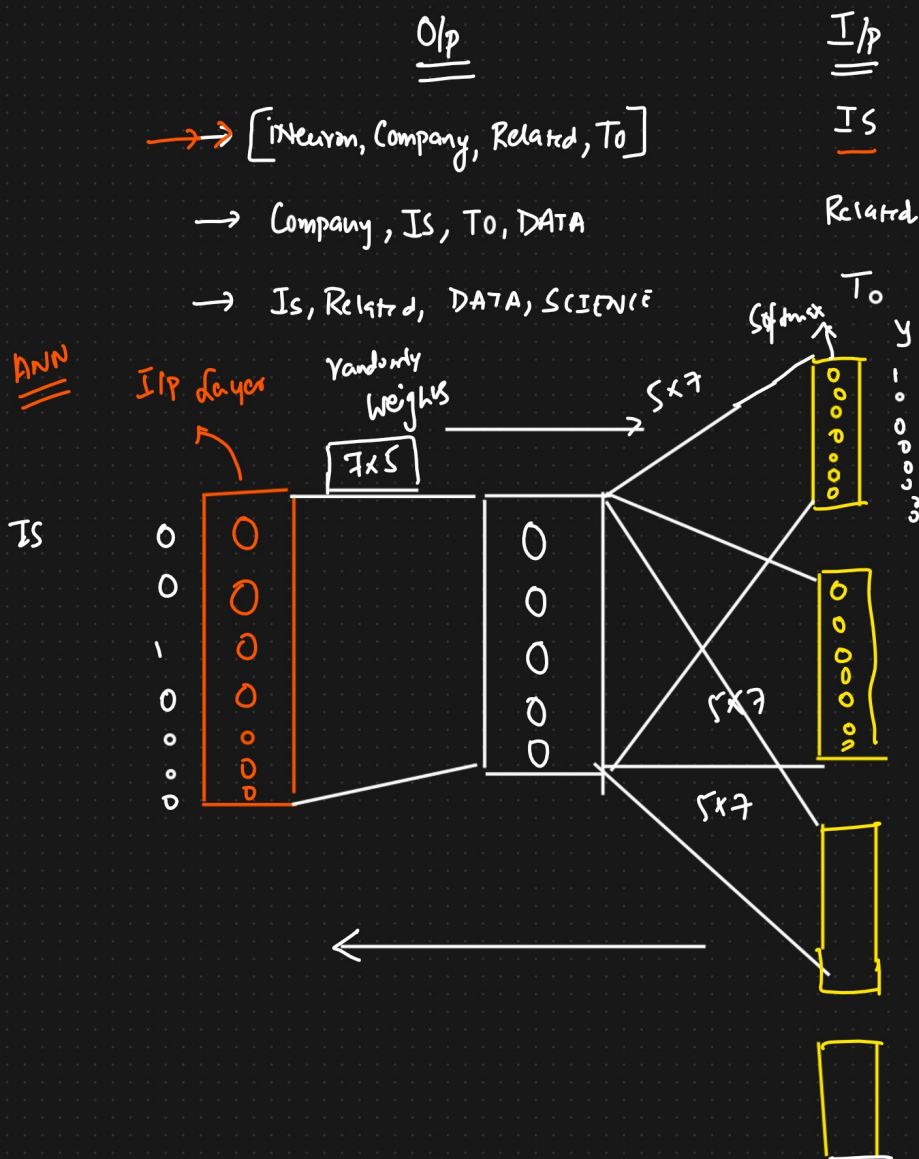
inNeuron [0.92, 0.44, 0.25, 0.36]
0.45

Company

[, , , , ,]

② Skipgram - Word2Vec

Window Size = 5



When Should we apply CBow or SkipGram.

{ Small Dataset → CBow
 Huge Dataset → SkipGram

① Increasing the Training Data

② Increase the window size
- vector dimension is also increasing.

Google Word2vec

Gensim

3 billion words → Google News

feature representation of 300 dimension vectors

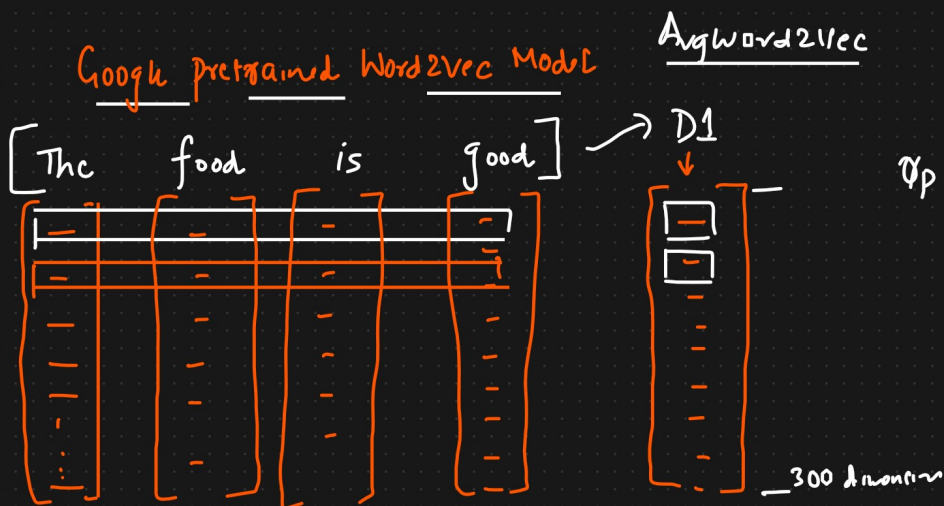
Cricket → [- - - - -]

Advantages of Word2Vec

- * Sparse Matrix $\longrightarrow$ Dense Matrix
- * Semantic Info is getting captured [Monet, good]
- * Vocabulary Size $\longrightarrow$ Fixed set of dimension vectors
Google Word2Vec [300 dimension]
- * OOV is also solved

* Avg Word2Vec

	<u>Text</u>	<u>O/P</u>	<u>Avg word2vec</u>	<u>O/P</u>
D1	The food is good	1	[- - - - -]	1
D2	The food is bad	0	[	]
D3	Pizza is Amazing	1	[	]



* Gensim, Glove

$\rightarrow$ pretrained Google Word2Vec

